

MCA Semester III

Advance Java Programming (Spring Boot)

Practical Assignment — Worked Solutions (Practicals 1-5)

Chapter 5 — Working with Spring Data JPA with MySQL Database

Each practical includes: Objective • Class/ER Diagram • Fully Commented Code • Step-by-Step Instructions • Postman API Testing

Before You Start: One-Time Project Setup

All 20 practicals live inside a single Spring Boot project so entities you build in one practical (e.g. Employee) can be reused by later practicals. Each practical gets its own Java sub-package (p1, p2, p3 ...) so identically-named classes in different practicals never collide.

Step 1 — Create the Spring Boot Project

1. Go to <https://start.spring.io> (Spring Initializr).
2. Project: Maven, Language: Java, Spring Boot: 3.3.x
3. Group: com.exam Artifact: spring-practical-exam Packaging: Jar Java: 17
4. Add dependencies: Spring Web, Spring Data JPA, MySQL Driver, Validation, Spring Security, Lombok.
5. Click Generate, unzip the project, and open it in IntelliJ IDEA / Eclipse / VS Code.

***i** You can also add dependencies later by editing pom.xml directly (shown below) instead of re-generating the project.*

Step 1a — Install MySQL and create the schema

This project connects to a real MySQL Server, so install MySQL Server + MySQL Workbench (or use the mysql CLI) first, then create an empty schema — Hibernate will create every table inside it automatically.

Run once in MySQL Workbench / mysql CLI

```
CREATE DATABASE IF NOT EXISTS examdb;
```

***i** The datasource URL below also includes createDatabaseIfNotExist=true as a safety net, but creating the schema explicitly first is recommended so you can confirm your MySQL credentials work before running Spring Boot.*

Step 2 — pom.xml (all dependencies used across Practicals 1-20)

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <groupId>com.exam</groupId>
  <artifactId>spring-practical-exam</artifactId>
  <version>1.0.0</version>
  <name>spring-practical-exam</name>
  <description>MCA Sem III - Spring Boot Practical Exam (Practicals 1-20)</description>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <!-- Core REST + MVC support (Chapter 6) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
```

```

<!-- Spring Data JPA (Chapter 5) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL JDBC driver -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Bean Validation - needed from Practical 13 onward -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Spring Security - needed from Practical 15 onward -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

Step 3 — application.properties

Replace `spring.datasource.username` / `spring.datasource.password` below with your own MySQL credentials. Every entity/repository/controller written in this document runs unchanged against MySQL — only these connection settings are MySQL-specific.

src/main/resources/application.properties

```

# ==== Server ====
server.port=8080

# ==== MySQL datasource ====
# Make sure MySQL Server is installed and running, and that the schema
# below already exists (see Step 1a) before you start the application.
spring.datasource.url=jdbc:mysql://localhost:3306/examdb?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver

```

```

spring.datasource.username=root
spring.datasource.password=your_mysql_password

# ==== JPA / Hibernate ====
# ddl-auto=update -> Hibernate creates/updates tables from @Entity classes automatically
spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

Step 4 — Recommended Package Layout

Project structure	
spring-practical-exam	
└─ src/main/java/com/exam	
└─ SpringPracticalExamApplication.java	
└─ p1/ Employee, EmployeeRepository, EmployeeRestController	
└─ p2/ Product, ProductRepository, ProductRestController	
└─ p3/ (reuses p1.Employee) EmployeeRepository (custom queries)	
└─ p4/ User, UserProfile, Employee, Passport, Country, Capital ...	
└─ p5/ Department, Employee, DepartmentRepository, DepartmentRestController	
└─ p6/ Student, Course ...	
└─ ... (one sub-package per practical, p6 - p20)	
└─ p20/ CorsConfig ...	
└─ src/main/resources/application.properties	
└─ pom.xml	

Step 5 — Running the Application

1. From your IDE, right-click SpringPracticalExamApplication.java → Run.
2. Or from a terminal in the project folder, run: `mvnw spring-boot:run` (Windows) or `./mvnw spring-boot:run` (Mac/Linux).
3. Confirm the console shows: Tomcat started on port(s): 8080 and Started SpringPracticalExamApplication.
4. Open MySQL Workbench, refresh the examdb schema, and confirm new tables appear as each practical's entities are added (Hibernate creates them on first run).
5. Open Postman (download from <https://www.postman.com/downloads/> if not already installed) — this is what we use to test every REST endpoint below.

Practical No. 1 — Introduction to JPA: Entity Class and Persisting a Single Entity

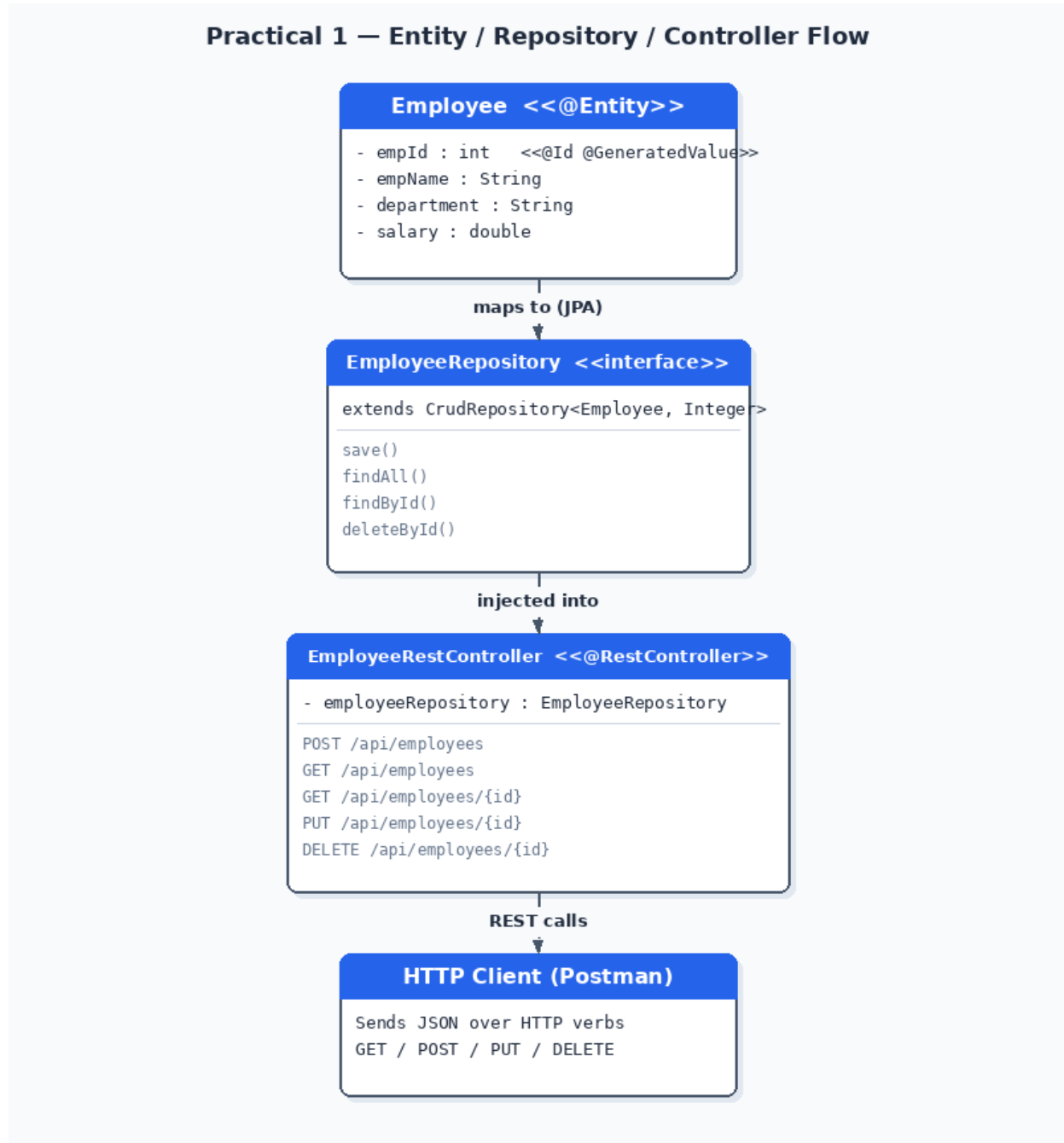
Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To understand the requirements of a JPA entity class (no-arg constructor, @Entity, @Id, @GeneratedValue), configure a relational datasource in application.properties, and persist/retrieve an entity using Spring Data JPA's CrudRepository.

Design — Class Diagram

The flow below is the pattern every practical in this document follows: an `@Entity` class is mapped to a table, a Repository interface gives us ready-made database operations, and a `@RestController` exposes those operations over HTTP for Postman (or any client) to call.



Step-by-Step Implementation

Step 1 — Add the dependencies (already done in the common pom.xml)

spring-boot-starter-data-jpa and the MySQL driver were added in the one-time setup section above. spring-boot-starter-data-jpa auto-configures a DataSource, an EntityManagerFactory and transaction management for us.

Step 2 — Configure the datasource (already done in application.properties)

spring.jpa.hibernate.ddl-auto=update tells Hibernate: read every @Entity class on startup and create or alter the matching table automatically, so we never hand-write CREATE TABLE statements during development.

Step 3 — Create the Employee entity

Create the file com.exam.p1.Employee. Key annotations:

- @Entity — marks the class as a JPA-managed table.
- @Id — marks empId as the primary key column.
- @GeneratedValue(strategy = GenerationType.IDENTITY) — the database itself auto-increments the id on insert.

src/main/java/com/exam/p1/Employee.java

```
package com.exam.p1;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

/**
 * Employee entity.
 * @Entity tells Hibernate this class maps to a database table ("employee").
 * A no-arg constructor is required by JPA so Hibernate can instantiate the
 * object via reflection before populating its fields.
 */
@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // DB auto-increments empId
    private int empId;

    private String empName;
    private String department;
    private double salary;

    // ---- No-arg constructor (REQUIRED by JPA) ----
    public Employee() {
    }

    // ---- Convenience constructor for creating objects in code ----
    public Employee(String empName, String department, double salary) {
        this.empName = empName;
        this.department = department;
        this.salary = salary;
    }

    // ---- Getters and Setters (JPA + Jackson JSON binding need these) ----
    public int getEmpId() {
        return empId;
    }

    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }
}
```

```

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }
}

```

Step 4 — Create EmployeeRepository

This interface needs no method bodies — Spring Data JPA generates the implementation at runtime using a dynamic proxy.

src/main/java/com/exam/p1/EmployeeRepository.java

```

package com.exam.p1;

import org.springframework.data.repository.CrudRepository;

/**
 * EmployeeRepository
 * Extending CrudRepository<Employee, Integer> instantly gives us, with
 * ZERO implementation code, the following methods (Spring generates a
 * proxy implementation at runtime):
 * save(Employee)          -> INSERT or UPDATE
 * findAll()               -> SELECT * FROM employee
 * findById(Integer)       -> SELECT * FROM employee WHERE emp_id = ?
 * deleteById(Integer)     -> DELETE FROM employee WHERE emp_id = ?
 * count(), existsById(), ... etc.
 *
 * Generic types: <Employee, Integer> = <Entity type, Primary-key type>
 */
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {
}

```

Step 5 & 6 — Create EmployeeRestController with full CRUD

@RestController combines @Controller + @ResponseBody, so every method's return value is automatically serialized to JSON. @RequestMapping("/api/employees") sets the common base path for all endpoints in this class.

src/main/java/com/exam/p1/EmployeeRestController.java

```

package com.exam.p1;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

/**
 * EmployeeRestController
 * Exposes the Employee entity over HTTP as a REST API.
 * Base path for every method below: /api/employees
 */

```

```

*/
@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository employeeRepository;

    // ---- CREATE : POST /api/employees ----
    @PostMapping
    public Employee createEmployee(@RequestBody Employee employee) {
        // save() inserts a new row and returns the row WITH the generated empId
        return employeeRepository.save(employee);
    }

    // ---- READ ALL : GET /api/employees ----
    @GetMapping
    public List<Employee> getAllEmployees() {
        // CrudRepository.findAll() returns Iterable<Employee>; wrap into a List
        List<Employee> list = new java.util.ArrayList<>();
        employeeRepository.findAll().forEach(list::add);
        return list;
    }

    // ---- READ ONE : GET /api/employees/{id} ----
    @GetMapping("/{id}")
    public Employee getEmployeeById(@PathVariable int id) {
        Optional<Employee> employee = employeeRepository.findById(id);
        return employee.orElse(null); // Practical 14 replaces this with a proper 404
    }

    // ---- UPDATE : PUT /api/employees/{id} ----
    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable int id, @RequestBody Employee updated) {
        // 1) fetch the existing row
        Optional<Employee> existingOpt = employeeRepository.findById(id);
        if (existingOpt.isEmpty()) {
            return null;
        }
        // 2) modify its fields
        Employee existing = existingOpt.get();
        existing.setEmpName(updated.getEmpName());
        existing.setDepartment(updated.getDepartment());
        existing.setSalary(updated.getSalary());
        // 3) save() again -> id already exists, so Hibernate runs UPDATE, not INSERT
        return employeeRepository.save(existing);
    }

    // ---- DELETE : DELETE /api/employees/{id} ----
    @DeleteMapping("/{id}")
    public String deleteEmployee(@PathVariable int id) {
        employeeRepository.deleteById(id);
        return "Employee deleted successfully";
    }
}

```

Step 7 — Run and verify

1. Run SpringPracticalExamApplication. Watch the console log — Hibernate prints "create table employee (...)" the first time it runs.
2. Open MySQL Workbench, connect to your local server, and confirm an employee table now exists inside the examdb schema (it will be empty).
3. Use Postman (next section) to insert a record with POST, then re-run `SELECT * FROM employee;` in Workbench (or the mysql CLI) to see the row physically stored.

Postman API Testing

With the application running on port 8080, open Postman and test each endpoint below, in order. Set the Body tab to raw / JSON for POST and PUT requests.

Practical 1 — Postman API Testing (Employee CRUD)

► 1. POST — create a new employee

POST http://localhost:8080/api/employees **Send**

Body > raw (JSON)

```
{
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 55000.0
}
```

Response **Status: 200 OK**

```
{
  "empId": 1,
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 55000.0
}
```

► 2. GET — fetch all employees

GET http://localhost:8080/api/employees **Send**

Response **Status: 200 OK**

```
[
  {
    "empId": 1,
    "empName": "Rahul Sharma",
    "department": "IT",
    "salary": 55000.0
  }
]
```

► 3. GET — fetch employee by id

GET http://localhost:8080/api/employees/1 **Send**

Response **Status: 200 OK**

```
{
  "empId": 1,
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 55000.0
}
```

► 4. PUT — update employee salary

PUT http://localhost:8080/api/employees/1 **Send**

Body > raw (JSON)

```
{
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 62000.0
}
```

Response **Status: 200 OK**

```
{
  "empId": 1,
  "empName": "Rahul Sharma",
```

i Tip: after the POST call returns emplId: 1, reuse that same id (1) in the GET-by-id, PUT and DELETE calls that follow.

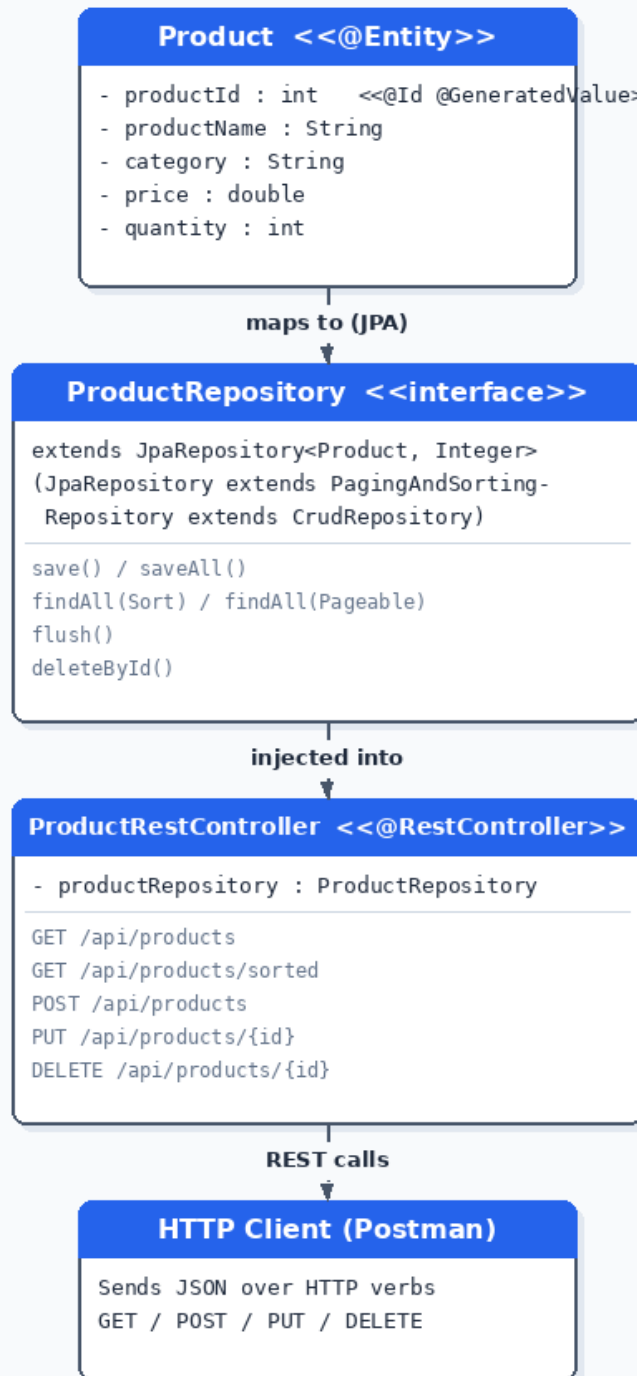
Practical No. 2 — Spring Data Repository Hierarchy: CrudRepository vs PagingAndSortingRepository vs JpaRepository

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To compare the three levels of the Spring Data repository hierarchy — CrudRepository (basic CRUD), PagingAndSortingRepository (adds paging/sorting), and JpaRepository (adds JPA-specific batch operations) — by rewriting the same repository at each level and observing the additional methods available.

Design — Class Diagram

Practical 2 — Repository Hierarchy: Crud -> PagingAndSorting -> Jpa

Step-by-Step Implementation

Step 1 — Create the Product entity

```
src/main/java/com/exam/p2/Product.java
package com.exam.p2;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int productId;

    private String productName;
    private String category;
    private double price;
    private int quantity;

    public Product() {
    }

    public Product(String productName, String category, double price, int quantity) {
        this.productName = productName;
        this.category = category;
        this.price = price;
        this.quantity = quantity;
    }

    // Getters and Setters
    public int getProductId() { return productId; }
    public void setProductId(int productId) { this.productId = productId; }
    public String getProductName() { return productName; }
    public void setProductName(String productName) { this.productName = productName; }
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

Step 2 — Stage 1: extend CrudRepository

Write this first and expose save/findAll endpoints. At this stage only basic CRUD methods are available — there is no way to sort or page results.

```
ProductRepository.java (Stage 1)
// ----- STAGE 1 : CrudRepository -----
// Gives only basic CRUD: save(), findAll(), findById(), deleteById(), count()
public interface ProductRepository extends CrudRepository<Product, Integer> {
}
```

Step 3 — Stage 2: change to PagingAndSortingRepository

Simply change the extends clause. Immediately, `findAll(Sort)` and `findAll(Pageable)` become available on the same interface — no other code changes.

ProductRepository.java (Stage 2)

```
// ----- STAGE 2 : PagingAndSortingRepository -----
// Adds on top of CrudRepository:
//   findAll(Sort sort)      -> get all rows, sorted
//   findAll(Pageable pageable) -> get one page of rows
public interface ProductRepository extends PagingAndSortingRepository<Product, Integer> {
}
```

Step 4 — Stage 3 (final): change to JpaRepository

`JpaRepository` sits at the top of the hierarchy: `JpaRepository` → `PagingAndSortingRepository` → `CrudRepository`. Switching to it keeps every method from the previous two stages and adds JPA-specific batch operations such as `saveAll()` and `flush()`.

src/main/java/com/exam/p2/ProductRepository.java (FINAL)

```
package com.exam.p2;

import org.springframework.data.jpa.repository.JpaRepository;

/**
 * ----- STAGE 3 (FINAL) : JpaRepository -----
 * JpaRepository extends PagingAndSortingRepository, which itself extends
 * CrudRepository. So this single interface now carries ALL THREE levels:
 *
 * From CrudRepository:      save(), findById(), deleteById() ...
 * From PagingAndSortingRepository: findAll(Sort), findAll(Pageable)
 * NEW in JpaRepository:    saveAll(Iterable), flush(),
 *                          saveAndFlush(), deleteAllInBatch()
 *
 * In a real project you simply extend JpaRepository directly (as below);
 * Stages 1 and 2 above are shown only so you can see what becomes newly
 * available at each level of the hierarchy, as the practical asks.
 */
public interface ProductRepository extends JpaRepository<Product, Integer> {
}
```

Step 5 & 6 — Build ProductRestController exercising every stage

The controller below is written against the final `JpaRepository` version, but includes a dedicated `/sorted` endpoint (Stage 2 capability) and a `/batch` endpoint using `saveAll()+flush()` (Stage 3 capability), so all three hierarchy levels are demonstrably exercised.

src/main/java/com/exam/p2/ProductRestController.java

```
package com.exam.p2;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    @Autowired
    private ProductRepository productRepository;
```

```

// ---- CREATE : POST /api/products ----
@PostMapping
public Product createProduct(@RequestBody Product product) {
    return productRepository.save(product);
}

// ---- READ ALL : GET /api/products ----
@GetMapping
public List<Product> getAllProducts() {
    return productRepository.findAll(); // JpaRepository.findAll() returns List directly
}

// ---- READ ALL, SORTED : GET /api/products/sorted ----
// Demonstrates the method that PagingAndSortingRepository introduced.
@GetMapping("/sorted")
public List<Product> getProductsSortedByPrice() {
    return productRepository.findAll(Sort.by("price").ascending());
}

// ---- BATCH INSERT : POST /api/products/batch ----
// Demonstrates saveAll() + flush(), which are new in JpaRepository.
@PostMapping("/batch")
public List<Product> createProductsBatch(@RequestBody List<Product> products) {
    List<Product> saved = productRepository.saveAll(products);
    productRepository.flush(); // force Hibernate to write the batch to the DB immediately
    return saved;
}

// ---- UPDATE : PUT /api/products/{id} ----
@PutMapping("/{id}")
public Product updateProduct(@PathVariable int id, @RequestBody Product updated) {
    Optional<Product> existingOpt = productRepository.findById(id);
    if (existingOpt.isEmpty()) return null;
    Product existing = existingOpt.get();
    existing.setProductName(updated.getProductName());
    existing.setCategory(updated.getCategory());
    existing.setPrice(updated.getPrice());
    existing.setQuantity(updated.getQuantity());
    return productRepository.save(existing);
}

// ---- DELETE : DELETE /api/products/{id} ----
@DeleteMapping("/{id}")
public String deleteProduct(@PathVariable int id) {
    productRepository.deleteById(id);
    return "Product deleted successfully";
}
}

```

Step 7 — Test each version's endpoints in Postman before moving on

1. With Stage 1 code, test POST /api/products and GET /api/products only (sorting/paging endpoints would not compile yet).
2. Switch to Stage 2, add the /sorted endpoint, and re-test.
3. Switch to Stage 3 (final), add /batch, and run the complete Postman collection below.

i Because Hibernate auto-creates the table, switching the repository's extends clause between stages requires no manual database changes — just restart the app.

Postman API Testing

Practical 2 — Postman API Testing (Repository Hierarchy)**1. POST — create a product****POST**

http://localhost:8080/api/products

Send

Body > raw (JSON)

```
{
  "productName": "Laptop",
  "category": "Electronics",
  "price": 55000.0,
  "quantity": 10
}
```

Response

Status: 200 OK

```
{
  "productId": 1,
  "productName": "Laptop",
  "category": "Electronics",
  "price": 55000.0,
  "quantity": 10
}
```

2. GET — sorted by price (PagingAndSortingRepository)**GET**

http://localhost:8080/api/products/sorted

Send

Response

Status: 200 OK

```
[
  {
    "productId": 2,
    "productName": "Mouse",
    "category": "Electronics",
    "price": 499.0,
    "quantity": 50
  },
  {
    "productId": 1,
    "productName": "Laptop",
    "category": "Electronics",
    "price": 55000.0,
    "quantity": 10
  }
]
```

3. PUT — update a product**PUT**

http://localhost:8080/api/products/1

Send

Body > raw (JSON)

```
{
  "productName": "Laptop",
  "category": "Electronics",
  "price": 52999.0,
  "quantity": 8
}
```

Response

Status: 200 OK

```
{
  "productId": 1,
  "productName": "Laptop",
  "category": "Electronics",
  "price": 52999.0,
  "quantity": 8
}
```

4. DELETE — remove a product**DELETE**

http://localhost:8080/api/products/2

Send

Response

Status: 200 OK

```
{
  "message": "Product deleted successfully"
}
```

Practical No. 3 — Custom Query Methods: findBy, countBy and @Query (JPQL)

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To derive query methods automatically from method names (query derivation) and to write custom JPQL queries using @Query for requirements that cannot be expressed by naming conventions alone.

i This practical reuses the Employee entity and EmployeeRepository created in Practical 1 (package com.exam.p1) — we only ADD new method declarations and new controller endpoints; nothing from Practical 1 is deleted.

Design — Class Diagram



Step-by-Step Implementation

Step 1 & 2 — Add derived query methods to EmployeeRepository

Spring Data JPA parses a repository method's NAME at startup and builds the SQL for you — no implementation is written by hand. The keyword after "findBy"/"countBy" must match an entity field name exactly (case-sensitive to the property name).

- findByDepartment(String) → WHERE department = ?
- countByDepartment(String) → SELECT COUNT(*) ... WHERE department = ?
- findBySalaryGreaterThan(double) → WHERE salary > ?

Step 3 — Write a custom JPQL query with @Query

Some conditions (like a BETWEEN range with two bound parameters) are awkward to express as a method name, so we write the JPQL directly and bind parameters with @Param.

```
src/main/java/com/exam/p1/EmployeeRepository.java (extended)
package com.exam.p1;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.data.repository.query.Param;

import java.util.List;

/**
 * EmployeeRepository - extended with custom query methods.
 * (This is the SAME Employee / EmployeeRepository created in Practical 1 -
 * we simply add more method declarations to the interface.)
 */
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {

    // ---- DERIVED QUERY : Spring parses the method name and builds the SQL ----
    // "findBy" + "Department" -> SELECT * FROM employee WHERE department = ?
    List<Employee> findByDepartment(String department);

    // "countBy" + "Department" -> SELECT COUNT(*) FROM employee WHERE department = ?
    long countByDepartment(String department);

    // "findBy" + "Salary" + "GreaterThan" -> SELECT * FROM employee WHERE salary > ?
    List<Employee> findBySalaryGreaterThan(double salary);

    // ---- CUSTOM QUERY : hand-written JPQL for logic naming conventions can't express ----
    @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
    List<Employee> findBySalaryRange(@Param("min") double min, @Param("max") double max);
}
```

Step 4 & 5 — Expose each method through the REST controller + seed data

A /seed endpoint is added purely to quickly populate a few employees across departments and salary bands for testing, so the salary-range and department queries have realistic data to return.

```
src/main/java/com/exam/p1/EmployeeRestController.java (extended)
package com.exam.p1;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
```

```

/**
 * New endpoints added to EmployeeRestController (Practical 1) to expose the
 * custom query methods declared in EmployeeRepository above.
 */
@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository employeeRepository;

    // ... POST / GET / GET-by-id / PUT / DELETE from Practical 1 stay unchanged ...

    // ---- Seed sample data across departments & salary bands ----
    @PostMapping("/seed")
    public List<Employee> seedEmployees() {
        employeeRepository.save(new Employee("Rahul Sharma", "IT", 55000));
        employeeRepository.save(new Employee("Sneha Patil", "IT", 61000));
        employeeRepository.save(new Employee("Aditya Rane", "HR", 42000));
        employeeRepository.save(new Employee("Priya Deshmukh", "Finance", 70000));
        List<Employee> all = new java.util.ArrayList<>();
        employeeRepository.findAll().forEach(all::add);
        return all;
    }

    // ---- Derived query : GET /api/employees/department/{department} ----
    @GetMapping("/department/{department}")
    public List<Employee> getByDepartment(@PathVariable String department) {
        return employeeRepository.findByDepartment(department);
    }

    // ---- Derived query : GET /api/employees/department/{department}/count ----
    @GetMapping("/department/{department}/count")
    public long countByDepartment(@PathVariable String department) {
        return employeeRepository.countByDepartment(department);
    }

    // ---- Derived query : GET /api/employees/salary/greater/{salary} ----
    @GetMapping("/salary/greater/{salary}")
    public List<Employee> getBySalaryGreaterThan(@PathVariable double salary) {
        return employeeRepository.findBySalaryGreaterThan(salary);
    }

    // ---- Custom JPQL query : GET /api/employees/salary-range?min=..&max=.. ----
    @GetMapping("/salary-range")
    public List<Employee> getBySalaryRange(
        @RequestParam double min, @RequestParam double max) {
        return employeeRepository.findBySalaryRange(min, max);
    }
}

```

Step 6 — Test each query endpoint from Postman

1. Call POST /api/employees/seed once to insert sample rows.
2. Call each GET endpoint below and manually verify the returned rows actually satisfy the condition (e.g. every row from /department/IT really has department = "IT").
3. Use PUT /api/employees/{id} (from Practical 1) to change an employee's salary, then re-call /salary-range to confirm the result set changes accordingly.
4. Use DELETE /api/employees/{id} to remove one employee, then re-call /department/{d}/count and confirm the count decreased by 1.

Postman API Testing

Practical 3 — Postman API Testing (Derived & @Query Methods)**▶ 1. GET — employees by department (derived query)**

GET

http://localhost:8080/api/employees/department/IT

Send

Response

Status: 200 OK

```
[
  {
    "empId": 1,
    "empName": "Rahul Sharma",
    "department": "IT",
    "salary": 55000.0
  },
  {
    "empId": 2,
    "empName": "Sneha Patil",
    "department": "IT",
    "salary": 61000.0
  }
]
```

▶ 2. GET — count employees in a department

GET

http://localhost:8080/api/employees/department/IT/count

Send

Response

Status: 200 OK

```
2
```

▶ 3. GET — employees with salary greater than X

GET

http://localhost:8080/api/employees/salary/greater/60000

Send

Response

Status: 200 OK

```
[
  {
    "empId": 2,
    "empName": "Sneha Patil",
    "department": "IT",
    "salary": 61000.0
  }
]
```

▶ 4. GET — custom JPQL @Query for salary range

GET

http://localhost:8080/api/employees/salary-range?min=50000&max=65000

Send

Response

Status: 200 OK

```
[
  {
    "empId": 1,
    "empName": "Rahul Sharma",
    "department": "IT",
    "salary": 55000.0
  },
  {
    "empId": 2,
    "empName": "Sneha Patil",
    "department": "IT",
    "salary": 61000.0
  }
]
```

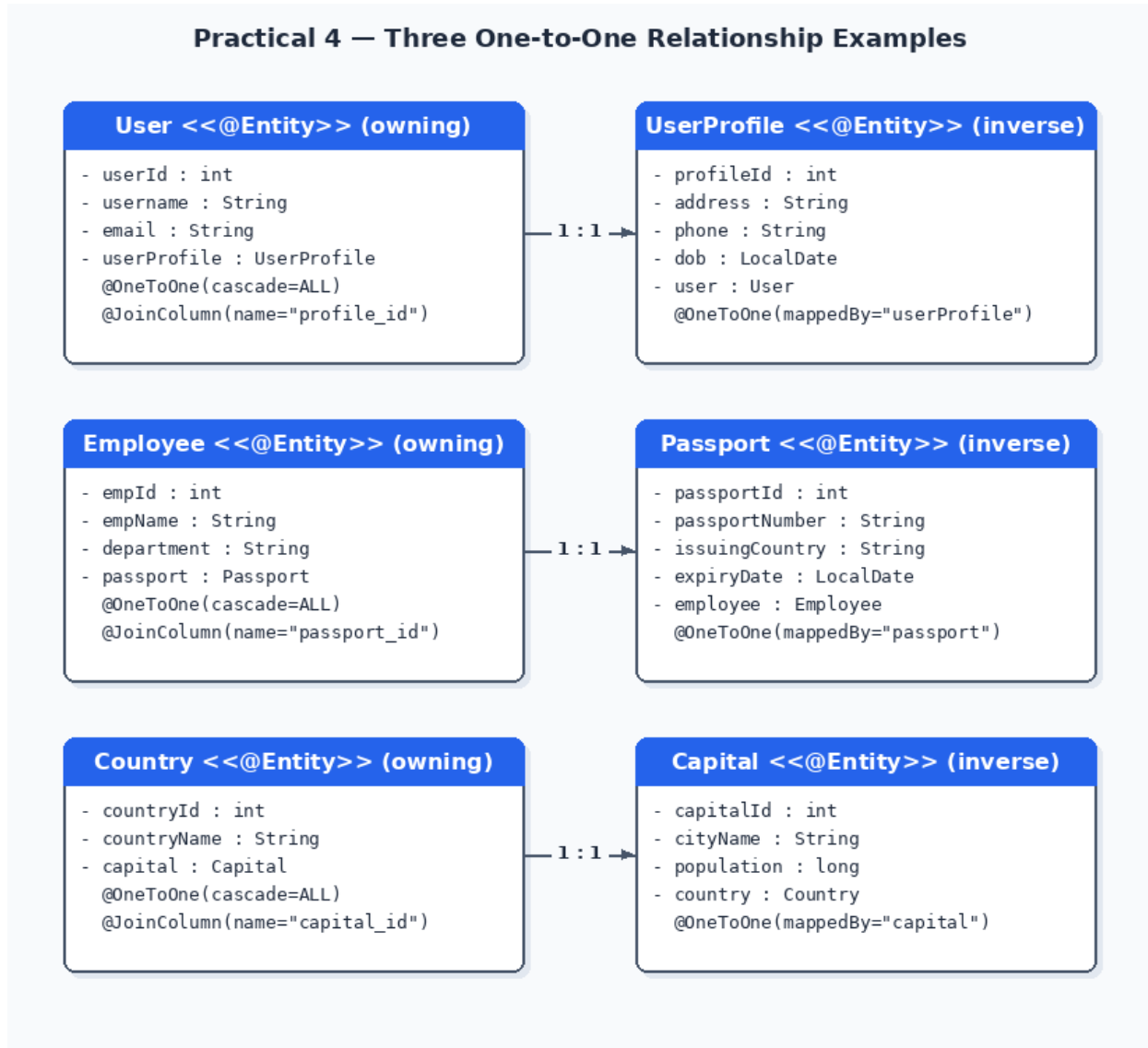
Practical No. 4 — One-to-One Mapping: Three Worked Examples

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To implement the One-to-One relationship using `@OneToOne`, `@JoinColumn` (owning side) and `mappedBy` (inverse side), with cascading so that saving a parent entity also persists its associated entity. Three independent pairs reinforce the pattern; full CRUD is built for Example 1 (User-UserProfile).

Design — Entity-Relationship Diagram (all three examples)



Key Concept: Owning Side vs Inverse Side

- The OWNING side (User, Employee, Country) carries the physical foreign-key column via `@JoinColumn`, and `cascade = CascadeType.ALL` on that side means saving/deleting the parent automatically saves/deletes the child.

- The INVERSE side (UserProfile, Passport, Capital) declares mappedBy = "<field name on owning side>" — it does NOT create a column; it only lets Java code navigate back from child to parent.

! @JsonIgnoreProperties is added on the owning side's relationship field purely to stop Jackson from serializing User -> profile -> user -> profile ... in an infinite loop when returning JSON.

Example 1 — User ↔ UserProfile (Full CRUD)

Step 1 — UserProfile entity (inverse side)

```
src/main/java/com/exam/p4/UserProfile.java
package com.exam.p4;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
public class UserProfile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int profileId;

    private String address;
    private String phone;
    private LocalDate dob;

    // Inverse side of the relationship: mappedBy points to the field name
    // ("UserProfile") on the OWNING entity (User). This side does NOT own
    // the foreign key column - it just lets Java navigate profile -> user.
    @OneToOne(mappedBy = "UserProfile")
    private User user;

    public UserProfile() {}

    public UserProfile(String address, String phone, LocalDate dob) {
        this.address = address;
        this.phone = phone;
        this.dob = dob;
    }

    public int getProfileId() { return profileId; }
    public void setProfileId(int profileId) { this.profileId = profileId; }
    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }
    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }
    public LocalDate getDob() { return dob; }
    public void setDob(LocalDate dob) { this.dob = dob; }
    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
}
```

Step 1 (contd.) — User entity (owning side)

```
src/main/java/com/exam/p4/User.java
package com.exam.p4;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
public class User {
```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int userId;

private String username;
private String email;

// Owning side: this entity's table holds the foreign key column
// "profile_id" (@JoinColumn). cascade = ALL means: whenever a User is
// saved/deleted, JPA automatically saves/deletes its UserProfile too -
// we never need to call userProfileRepository.save() separately.
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "profile_id")
@JsonIgnoreProperties("user") // avoid infinite User->profile->user->... JSON loop
private UserProfile userProfile;

public User() {}

public User(String username, String email, UserProfile userProfile) {
    this.username = username;
    this.email = email;
    this.userProfile = userProfile;
}

public int getUserId() { return userId; }
public void setUserId(int userId) { this.userId = userId; }
public String getUsername() { return username; }
public void setUsername(String username) { this.username = username; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public UserProfile getUserProfile() { return userProfile; }
public void setUserProfile(UserProfile userProfile) { this.userProfile = userProfile; }
}

```

Step 2 — UserRepository and UserRestController (full CRUD)

src/main/java/com/exam/p4/UserRepository.java

```

package com.exam.p4;

import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Integer> {
}

```

src/main/java/com/exam/p4/UserRestController.java

```

package com.exam.p4;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/users")
public class UserRestController {

    @Autowired
    private UserRepository userRepository;

    // POST /api/users - accepts nested profile JSON; cascade=ALL saves both rows
    @PostMapping

```

```

public User createUser(@RequestBody User user) {
    if (user.getUserProfile() != null) {
        user.getUserProfile().setUser(user); // keep both sides of the link in sync
    }
    return userRepository.save(user);
}

@GetMapping
public List<User> getAllUsers() {
    return userRepository.findAll();
}

@GetMapping("/{id}")
public User getUserById(@PathVariable int id) {
    return userRepository.findById(id).orElse(null);
}

// PUT /api/users/{id} - updates the user AND its nested profile
@PutMapping("/{id}")
public User updateUser(@PathVariable int id, @RequestBody User updated) {
    Optional<User> existingOpt = userRepository.findById(id);
    if (existingOpt.isEmpty()) return null;
    User existing = existingOpt.get();
    existing.setUsername(updated.getUsername());
    existing.setEmail(updated.getEmail());
    if (updated.getUserProfile() != null && existing.getUserProfile() != null) {
        existing.getUserProfile().setAddress(updated.getUserProfile().getAddress());
        existing.getUserProfile().setPhone(updated.getUserProfile().getPhone());
        existing.getUserProfile().setDob(updated.getUserProfile().getDob());
    }
    return userRepository.save(existing);
}

// DELETE /api/users/{id} - cascade = ALL removes the linked UserProfile too
@DeleteMapping("/{id}")
public String deleteUser(@PathVariable int id) {
    userRepository.deleteById(id);
    return "User (and linked profile) deleted successfully";
}
}

```

Example 2 — Employee ↔ Passport (same pattern)

Step 3: repeat the identical owning/inverse pattern under /api/employees-passport.

src/main/java/com/exam/p4/Passport.java

```

package com.exam.p4;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
public class Passport {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int passportId;

    private String passportNumber;
    private String issuingCountry;
    private LocalDate expiryDate;

    @OneToOne(mappedBy = "passport")
    private Employee employee;
}

```

```

public Passport() {}
public Passport(String passportNumber, String issuingCountry, LocalDate expiryDate) {
    this.passportNumber = passportNumber;
    this.issuingCountry = issuingCountry;
    this.expiryDate = expiryDate;
}
// Getters & Setters
public int getPassportId() { return passportId; }
public void setPassportId(int passportId) { this.passportId = passportId; }
public String getPassportNumber() { return passportNumber; }
public void setPassportNumber(String passportNumber) { this.passportNumber = passportNumber; }
public String getIssuingCountry() { return issuingCountry; }
public void setIssuingCountry(String issuingCountry) { this.issuingCountry = issuingCountry; }
public LocalDate getExpiryDate() { return expiryDate; }
public void setExpiryDate(LocalDate expiryDate) { this.expiryDate = expiryDate; }
public Employee getEmployee() { return employee; }
public void setEmployee(Employee employee) { this.employee = employee; }
}

```

src/main/java/com/exam/p4/Employee.java

```

package com.exam.p4;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private String department;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "passport_id")
    @JsonIgnoreProperties("employee")
    private Passport passport;

    public Employee() {}
    public Employee(String empName, String department, Passport passport) {
        this.empName = empName; this.department = department; this.passport = passport;
    }
    public int getEmpId() { return empId; }
    public void setEmpId(int empId) { this.empId = empId; }
    public String getEmpName() { return empName; }
    public void setEmpName(String empName) { this.empName = empName; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
    public Passport getPassport() { return passport; }
    public void setPassport(Passport passport) { this.passport = passport; }
}

```

src/main/java/com/exam/p4/EmployeePassportController.java

```

package com.exam.p4;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

interface EmployeePassportRepository extends JpaRepository<Employee, Integer> {}

```

```

@RestController
@RequestMapping("/api/employees-passport")
class EmployeePassportController {

    @Autowired
    private EmployeePassportRepository employeeRepository;

    // POST /api/employees-passport - create employee with nested passport
    @PostMapping
    public Employee create(@RequestBody Employee employee) {
        if (employee.getPassport() != null) {
            employee.getPassport().setEmployee(employee);
        }
        return employeeRepository.save(employee);
    }

    // GET /api/employees-passport - list all, each with its passport
    @GetMapping
    public List<Employee> getAll() {
        return employeeRepository.findAll();
    }
}

```

Example 3 — Country ↔ Capital (same pattern)

Step 4: repeat the pattern once more under /api/countries.

src/main/java/com/exam/p4/Country.java

```

package com.exam.p4;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
public class Country {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int countryId;

    private String countryName;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "capital_id")
    @JsonIgnoreProperties("country")
    private Capital capital;

    public Country() {}
    public Country(String countryName, Capital capital) {
        this.countryName = countryName; this.capital = capital;
    }
    public int getCountryId() { return countryId; }
    public void setCountryId(int countryId) { this.countryId = countryId; }
    public String getCountryName() { return countryName; }
    public void setCountryName(String countryName) { this.countryName = countryName; }
    public Capital getCapital() { return capital; }
    public void setCapital(Capital capital) { this.capital = capital; }
}

```

src/main/java/com/exam/p4/Capital.java

```

package com.exam.p4;

import jakarta.persistence.*;

@Entity
public class Capital {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int capitalId;

    private String cityName;
    private long population;

    @OneToOne(mappedBy = "capital")
    private Country country;

    public Capital() {}
    public Capital(String cityName, long population) {
        this.cityName = cityName; this.population = population;
    }
    public int getCapitalId() { return capitalId; }
    public void setCapitalId(int capitalId) { this.capitalId = capitalId; }
    public String getCityName() { return cityName; }
    public void setCityName(String cityName) { this.cityName = cityName; }
    public long getPopulation() { return population; }
    public void setPopulation(long population) { this.population = population; }
    public Country getCountry() { return country; }
    public void setCountry(Country country) { this.country = country; }
}

```

src/main/java/com/exam/p4/CountryController.java

```

package com.exam.p4;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

interface CountryRepository extends JpaRepository<Country, Integer> {}

@RestController
@RequestMapping("/api/countries")
class CountryController {

    @Autowired
    private CountryRepository countryRepository;

    // POST /api/countries - create country with nested capital
    @PostMapping
    public Country create(@RequestBody Country country) {
        if (country.getCapital() != null) {
            country.getCapital().setCountry(country);
        }
        return countryRepository.save(country);
    }

    // GET /api/countries - list all countries with their capitals
    @GetMapping
    public List<Country> getAll() {
        return countryRepository.findAll();
    }
}

```

Step 5 & 6 — Verify cascading and test in Postman

1. Run full CRUD (POST, GET, PUT, DELETE) against Example 1 in Postman, in that order.
2. For Examples 2 and 3, at minimum repeat POST and GET to confirm the same cascading pattern generalizes.
3. After a DELETE on `/api/users/{id}`, check MySQL Workbench: the linked row in `user_profile` should also be gone, proving cascade = ALL removed the child too.

Postman API Testing

Practical 4 — Postman API Testing (One-to-One x3)**► 1. POST — create User with nested UserProfile (cascade save)**

POST http://localhost:8080/api/users **Send**

Body > raw (JSON)

```
{
  "username": "amit01",
  "email": "amit01@example.com",
  "userProfile": {
    "address": "Pune, MH",
    "phone": "9876543210",
    "dob": "1998-05-14"
  }
}
```

Response **Status: 200 OK**

```
{
  "userId": 1,
  "username": "amit01",
  "email": "amit01@example.com",
  "userProfile": {
    "profileId": 1,
    "address": "Pune, MH",
    "phone": "9876543210",
    "dob": "1998-05-14"
  }
}
```

► 2. GET — fetch user with profile

GET http://localhost:8080/api/users/1 **Send**

Response **Status: 200 OK**

```
{
  "userId": 1,
  "username": "amit01",
  "email": "amit01@example.com",
  "userProfile": {
    "profileId": 1,
    "address": "Pune, MH",
    "phone": "9876543210",
    "dob": "1998-05-14"
  }
}
```

► 3. POST — Example 2: Employee with nested Passport

POST http://localhost:8080/api/employees-passport **Send**

Body > raw (JSON)

```
{
  "empName": "Kiran Rao",
  "department": "HR",
  "passport": {
    "passportNumber": "P1234567",
    "issuingCountry": "India",
    "expiryDate": "2030-01-01"
  }
}
```

Response **Status: 200 OK**

```
{
  "empId": 1,
  "empName": "Kiran Rao",
  "department": "HR",
  "passport": {
    "passportId": 1,
    "passportNumber": "P1234567",
    "issuingCountry": "India",
    "expiryDate": "2030-01-01"
  }
}
```

► 4. POST — Example 3: Country with nested Capital

POST http://localhost:8080/api/countries **Send**

Body > raw (JSON)

```
{
  "countryName": "India",
  "capital": {
    "cityName": "New Delhi",
    "population": 32900000
  }
}
```

Response **Status: 200 OK**

```
{
  "countryId": 1,
  "countryName": "India",
  "capital": {
    "capitalId": 1,
    "cityName": "New Delhi",
    "population": 32900000
  }
}
```

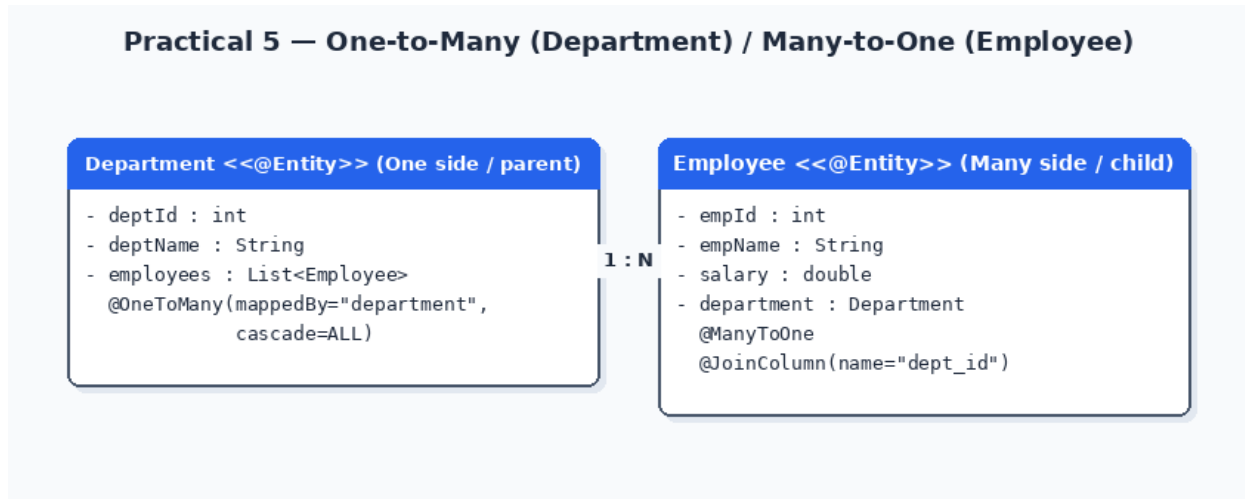
Practical No. 5 — One-to-Many / Many-to-One Mapping: Department and Employee

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To implement a bidirectional One-to-Many (Department → Employees) and Many-to-One (Employee → Department) relationship using `mappedBy` on the parent side and `@JoinColumn` on the child side, and to persist a parent with a list of children in a single request.

Design — Entity-Relationship Diagram



Step-by-Step Implementation

Step 1 — Employee entity (the "Many" side)

Employee owns the foreign key: its table gets a `dept_id` column pointing back to Department.

```

src/main/java/com/exam/p5/Employee.java
package com.exam.p5;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private double salary;

    // MANY side: this table stores the foreign key column "dept_id".
    // Many Employee rows can point to the same Department.
    @ManyToOne
    @JoinColumn(name = "dept_id")
    @JsonIgnoreProperties("employees") // stop Department -> employees -> department -> ... loop
    private Department department;
  
```

```

public Employee() {}
public Employee(String empName, double salary) {
    this.empName = empName;
    this.salary = salary;
}

public int getEmpId() { return empId; }
public void setEmpId(int empId) { this.empId = empId; }
public String getEmpName() { return empName; }
public void setEmpName(String empName) { this.empName = empName; }
public double getSalary() { return salary; }
public void setSalary(double salary) { this.salary = salary; }
public Department getDepartment() { return department; }
public void setDepartment(Department department) { this.department = department; }
}

```

Step 2 — Department entity (the "One" side)

mappedBy = "department" tells Hibernate the relationship is already described on the Employee side — Department does not get its own foreign-key column. orphanRemoval = true additionally allows deleting a child by simply removing it from this list.

src/main/java/com/exam/p5/Department.java

```

package com.exam.p5;

import jakarta.persistence.*;
import java.util.List;

@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int deptId;

    private String deptName;

    // ONE side: mappedBy = "department" points at the field name on the
    // MANY (Employee) side. cascade = ALL means saving a Department with a
    // populated employees list also inserts every Employee row.
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Employee> employees;

    public Department() {}
    public Department(String deptName, List<Employee> employees) {
        this.deptName = deptName;
        this.employees = employees;
    }

    public int getDeptId() { return deptId; }
    public void setDeptId(int deptId) { this.deptId = deptId; }
    public String getDeptName() { return deptName; }
    public void setDeptName(String deptName) { this.deptName = deptName; }
    public List<Employee> getEmployees() { return employees; }
    public void setEmployees(List<Employee> employees) { this.employees = employees; }
}

```

Step 3 — DepartmentRepository

```
src/main/java/com/exam/p5/DepartmentRepository.java
```

```
package com.exam.p5;

import org.springframework.data.jpa.repository.JpaRepository;

public interface DepartmentRepository extends JpaRepository<Department, Integer> {
}
```

Step 4-7 — DepartmentRestController (full CRUD, both directions)

Notice the important line in `createDepartment()`: each child `Employee`'s `setDepartment(department)` must be called before saving, or the `dept_id` foreign key would be written as `NULL` — `cascade` only tells JPA to also `INSERT` the child rows, it does not set the back-reference for you.

```
src/main/java/com/exam/p5/DepartmentRestController.java
```

```
package com.exam.p5;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/departments")
public class DepartmentRestController {

    @Autowired
    private DepartmentRepository departmentRepository;

    // ---- CREATE : POST /api/departments ----
    // Accepts a department name + list of employee objects, saves the whole
    // graph in one call thanks to cascade = ALL on Department.employees.
    @PostMapping
    public Department createDepartment(@RequestBody Department department) {
        if (department.getEmployees() != null) {
            // IMPORTANT: set the back-reference on every child before saving,
            // otherwise the dept_id foreign-key column would be left NULL.
            department.getEmployees().forEach(emp -> emp.setDepartment(department));
        }
        return departmentRepository.save(department);
    }

    // ---- READ : GET /api/departments/{id} ----
    @GetMapping("/{id}")
    public Department getDepartment(@PathVariable int id) {
        return departmentRepository.findById(id).orElse(null);
    }

    // ---- READ only the employee list : GET /api/departments/{id}/employees ----
    @GetMapping("/{id}/employees")
    public List<Employee> getEmployeesOfDepartment(@PathVariable int id) {
        Optional<Department> dept = departmentRepository.findById(id);
        return dept.map(Department::getEmployees).orElse(null);
    }

    // ---- RENAME department : PUT /api/departments/{id} ----
    @PutMapping("/{id}")
    public Department renameDepartment(@PathVariable int id, @RequestBody Department updated) {
        Optional<Department> existingOpt = departmentRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Department existing = existingOpt.get();
        existing.setDeptName(updated.getDeptName());
    }
}
```

```

        return departmentRepository.save(existing);
    }

    // ---- UPDATE employee inside a department : PUT .../{deptId}/employees/{empId} ----
    // Demonstrates Many-to-One navigation from Employee back to Department.
    @PutMapping("/{deptId}/employees/{empId}")
    public Employee updateEmployeeInDepartment(
        @PathVariable int deptId, @PathVariable int empId, @RequestBody Employee updated) {
        Optional<Department> deptOpt = departmentRepository.findById(deptId);
        if (deptOpt.isEmpty()) return null;
        Department department = deptOpt.get();
        for (Employee emp : department.getEmployees()) {
            if (emp.getEmpId() == empId) {
                emp.setEmpName(updated.getEmpName());
                emp.setSalary(updated.getSalary());
                departmentRepository.save(department); // cascades change to Employee row
                return emp;
            }
        }
        return null;
    }

    // ---- DELETE one employee from a department : DELETE .../{deptId}/employees/{empId} ----
    @DeleteMapping("/{deptId}/employees/{empId}")
    public String removeEmployeeFromDepartment(
        @PathVariable int deptId, @PathVariable int empId) {
        Optional<Department> deptOpt = departmentRepository.findById(deptId);
        if (deptOpt.isEmpty()) return "Department not found";
        Department department = deptOpt.get();
        // orphanRemoval = true on the @OneToMany means: removing an Employee
        // from this list and saving the parent DELETES that Employee's row.
        department.getEmployees().removeIf(emp -> emp.getEmpId() == empId);
        departmentRepository.save(department);
        return "Employee removed from department";
    }

    // ---- DELETE a whole department (+ all its employees, via cascade) ----
    @DeleteMapping("/{id}")
    public String deleteDepartment(@PathVariable int id) {
        departmentRepository.deleteById(id);
        return "Department and all its employees deleted successfully";
    }
}

```

Step 8 — Test in Postman

1. POST a department with a nested list of employees — confirm one call inserts both the department row and every employee row.
2. GET /api/departments/{id} and GET /api/departments/{id}/employees to confirm both read endpoints.
3. PUT /api/departments/{deptId}/employees/{empId} to update a single employee's salary from within the department.
4. DELETE one employee, then DELETE the whole department, and confirm via MySQL Workbench that cascading removed every related row.

Postman API Testing

Practical 5 — Postman API Testing (One-to-Many / Many-to-One)**1. POST — create Department with a list of Employees (cascade)**

POST http://localhost:8080/api/departments **Send**

Body > raw (JSON)

```
{
  "deptName": "Computer Science",
  "employees": [
    {
      "empName": "Neha Joshi",
      "salary": 48000.0
    },
    {
      "empName": "Vikas Nair",
      "salary": 52000.0
    }
  ]
}
```

Response **Status: 200 OK**

```
{
  "deptId": 1,
  "deptName": "Computer Science",
  "employees": [
    {
      "empId": 1,
      "empName": "Neha Joshi",
      "salary": 48000.0
    },
    {
      "empId": 2,
      "empName": "Vikas Nair",
      "salary": 52000.0
    }
  ]
}
```

2. GET — department with employees

GET http://localhost:8080/api/departments/1 **Send**

Response **Status: 200 OK**

```
{
  "deptId": 1,
  "deptName": "Computer Science",
  "employees": [
    {
      "empId": 1,
      "empName": "Neha Joshi",
      "salary": 48000.0
    },
    {
      "empId": 2,
      "empName": "Vikas Nair",
      "salary": 52000.0
    }
  ]
}
```

3. PUT — update one employee within a department

PUT http://localhost:8080/api/departments/1/employees/1 **Send**

Body > raw (JSON)

```
{
  "empName": "Neha Joshi",
  "salary": 51000.0
}
```

Response **Status: 200 OK**

```
{
  "empId": 1,
  "empName": "Neha Joshi",
  "salary": 51000.0,
  "department": {
    "deptId": 1,
    "deptName": "Computer Science"
  }
}
```

4. DELETE — remove one employee from a department

DELETE http://localhost:8080/api/departments/1/employees/2 **Send**

Response **Status: 200 OK**

```
{
  "message": "Employee removed from department"
}
```